Machine Learning Forecasting Engine (Cloud Run Microservice)

1. System Overview

We built a Python program hosted on Google Cloud that automatically runs every 30 minutes. It reads the most recent environmental data, trains a new AI model on the spot, predicts future conditions, and uploads those results to our Firestore database.

2. Data Scope and Forecast Horizon

- Historical Data Used (The Input): The model queries the last 20,000 readings from Firestore, which roughly equates to the last 14 days of historical data. This 2-week window is specifically chosen to allow the model to learn daily patterns (e.g., temperatures rising at noon) as well as weekly cycles (e.g., weekends having different pollution levels than weekdays).
- Forecast Timeframe (The Output): The model predicts exactly 7 days into the future.
- Resolution: Predictions are generated in 30-minute intervals, resulting in $7 \times 48 = 336$ distinct prediction steps per execution.

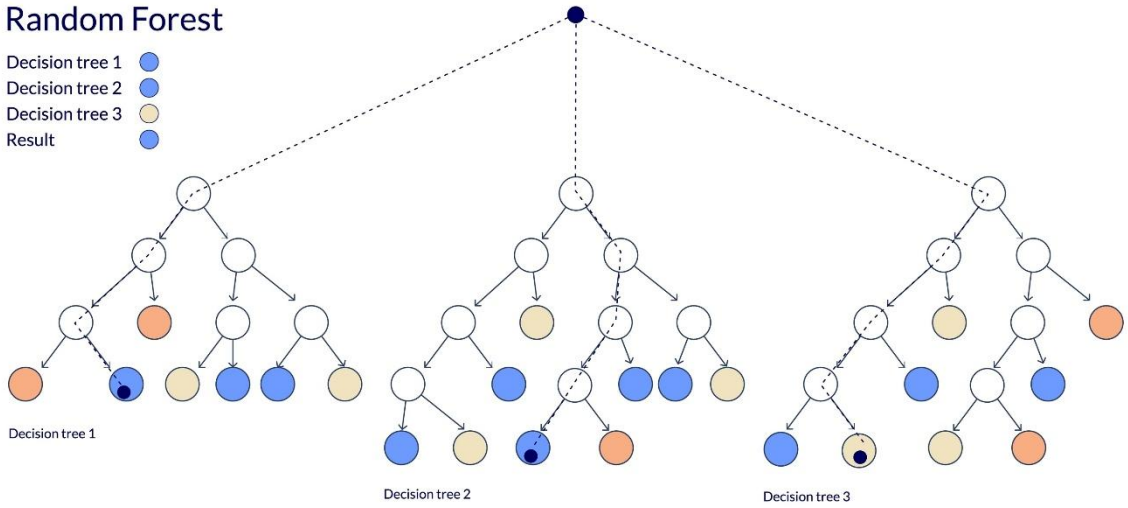
3. The Variables (Features & Targets)

The model analyzes and predicts seven distinct environmental variables simultaneously:

1) Temperature
2) Humidity
3) Dust Density
4) AQI (Air Quality Index)
5) TVOC (Total Volatile Organic Compounds)
6) CO2 (eCO2-Equivalent Carbon Dioxide)
7) Gas Pollution Percentage (from the MQ135)

Each prediction is generated with the data and the time.

4. The Chosen Algorithm: Random Forest Regression



The code uses Random Forest Regressor from the scikit-learn library as the model. Random Forest is an ensemble learning method that builds dozens of "Decision Trees" during training and averages their predictions to output a highly accurate final result.

Why Random Forest was chosen over other algorithms:

- Handling Non-Linearity: Environmental data is highly non-linear. Linear Regression fails here, but Random Forest handles non-linear spikes exceptionally well.
- Compute Efficiency for Cloud Run: While Deep Learning models like LSTMs (Long Short-Term Memory networks) are great for time-series, they require heavy computing power, GPUs, and long training times. Random Forest trains in a matter of seconds using standard CPUs, making it perfect for a lightweight, cost-effective Cloud Run environment that triggers every 30 minutes.
- No Data Scaling Required: Neural networks require all data to be strictly normalized (scaled between 0 and 1). Random Forests do not care about the scale of the data, meaning we can feed raw Temperature (e.g., 30°C) and raw eCO2 (e.g., 400ppm) into the exact same model without complex pre-processing.

5. The Crucial Mechanism: How the Model Predicts the Future

The most critical part of this architecture is how it handles the time-series forecasting. It uses a technique called **Autoregressive Recursive Forecasting**.

Here is the exact step-by-step mechanism the code follows to predict the future:

Step 1: Resampling the Data;

- Sensor data can be messy and irregular. To fix this, our system automatically put the raw data into neat 30-minute intervals for the 14 days, giving the AI a clean and consistent timeline to learn from.

Step 2: Training 7 Parallel Models;

The code does not train just one model. It initializes a loop and trains seven separate Random Forest models. One dedicated entirely to Temperature, one to AQI, one to Dust, etc. Each model learns how its specific variable behaves based on the previous 30-minute window of all combined variables.

The Training Phase (Using all 14 Days): The model uses the full 14 days of data, but only to study and learn the rules.

- When the time is 12:00 PM, the temperature usually goes up.
- When AQI is high at 2:00 PM, it usually stays high at 2:30 PM.
- Weekends have less CO2 than weekdays.

Once the AI finishes reading those 14 days, it builds its Random Forest "brain" and completely throws the historical data away.

Step 3: The Recursive Prediction Loop (The "Sliding Window")

To predict 7 days into the future, the code relies on a feedback loop:

1) The Starting Point: It takes the very last known real sensor reading (e.g., today at 12:00 PM).
2) Step 1 Prediction: It feeds that real data into the ML models to predict what the air will look like at 12:30 PM.
3) The Feedback Loop: To predict 1:00 PM, the model no longer has real data. Therefore, it takes its own 12:30 PM prediction and feeds it back into itself as the new input to generate the 1:00 PM prediction.
4) Iteration: This cycle repeats 336 times. The outputs of step N become the inputs for step N+1, chaining together to generate a full 7-day forecast.